



南京大學

NANJING UNIVERSITY

《面向 AI 生成代码的人工审核与智能支持》实验报告

学生姓名： 顾奕博、周楚然、尚昊毅、姚凯方

学生学号： 231250109、231250111、231250110、231250097

2026 年 5 月

1. 研究背景与动机

1.1 生成式 AI 重塑代码审核范式

代码评审是软件质量保障体系中不可或缺的一环。在传统的软件开发流程中，代码由开发人员逐行编写，评审者能够基于清晰的编写意图和上下文信息，对代码的正确性、可维护性和安全性做出判断。即便在引入静态分析工具（如 SonarQube、PMD）之后，代码审核的核心范式始终未发生根本性变化——工具负责规则匹配，人类负责意义理解与质量把关。然而，生成式人工智能的崛起正在动摇这一范式的基础。

近年来，以大型语言模型为核心的 AI 编程辅助工具（如 GitHub Copilot、DeepSeek Coder、CodeLlama 等）已从实验走向大规模应用 [3]。这些工具能够在秒级时间内根据自然语言描述或少量上下文代码，生成完整的函数、模块乃至系统架构。与传统人工编写相比，AI 生成代码的最显著特征不在于“正确性”本身，而在于其**生成方式与缺陷分布发生了结构性变化**：

- 从有序递进到碎片洪流**：AI 代码通常以高频率、细粒度的形式被源源不断地推送至代码仓库。传统代码评审假设的是“少量代码经过充分思考后提交”，而 AI 辅助场景下，同一开发者在一次提交中可能合入数百行由不同提示词驱动的代码片段。审核者面对的信息密度和复杂度远超以往。
- 从已知模式到未知风险**：静态分析工具的有效性依赖于缺陷模式的事先定义，但 AI 代码可能产生训练数据中未曾出现过的组合缺陷——代码表面语法正确、逻辑看似合理，却在特定边界条件下引发隐蔽错误。这类“合理的外观、危险的实质”给审核者带来了极高的判断难度。
- 从线性流程到并行压力**：交付节奏加快是 AI 赋能开发的初衷之一，但当代码生成速度大幅提升而审核人力有限时，“生成-审核”之间形成了时间上的剪刀差。审核环节若成为流水线瓶颈，团队往往面临两种困境：压缩审核时间导致质量下滑，或严格审核导致交付延迟。

在这一背景下，“**人类为最终代码质量签字确认**”这一原则并未改变，但签字的前提——即审核者做出判断所需的认知投入和信息支撑——已发生根本性动摇 [3]。

1.2 研究动机：构建 AI 时代人工审核的智能支撑体系

面对上述结构性挑战，学术界与工业界已开始探索将大模型和智能体技术引入代码审核流程的可行路径，例如：基于 LLM 的缺陷严重程度评分 [1]、图算法驱动的审核优先级排序 [2]、针对 AI 生成代码的修复方案生成等。这些

尝试展示了智能技术赋能审核的潜力，但当前研究多关注"技术能否检测出缺陷"，而较少关注"审核者实际需要什么样的帮助"。具体而言，当前研究存在以下不足：

- 1. **需求侧认知不足**：现有工作多从技术供给角度设计智能评审功能，较少系统调研真实开发者在日常审核中遭遇的核心痛点——他们究竟是被什么类型的 AI 代码缺陷所困扰？时间压力、认知负荷还是判断依据的缺失？
- 2. **信任机制尚不清晰**：开发者对智能评审工具的信任边界在哪里？是更信任 AI 给出的评审意见，还是更信任 AI 直接生成的修复代码？不同信任模式背后反映了什么样的人机协同预期？
- 3. **接受度影响因素待厘清**：智能评审工具能否真正落地，不仅取决于技术性能，还与开发者的经验背景、AI 工具使用习惯、对代码安全的顾虑等因素密切相关，但目前缺乏针对这些影响因素的系统性实证研究。

本研究认为，在"AI 辅助人审、人主导 AI 管"的新范式下，智能评审工具的设计不应是技术驱动的单向输出，而应建立在对人工审核实际需求深刻理解的基础之上。因此，本研究采用问卷调查方法，从开发者视角出发，系统收集 AI 生成代码场景下代码评审行为、人工审核成本感知以及智能评审支持需求与信任程度的第一手数据，为后续智能评审工具的针对性设计与有效评估提供实证依据。

2. 研究问题

基于上述背景与动机，本研究围绕”探究开发者在 AI 生成代码场景下的代码评审行为，以及对智能评审支持技术的需求与评价”这一核心目标，提出以下四个研究问题：

问题	类型
RQ1：AI 生成代码是否增加了代码评审负担？	影响评估
RQ2：开发者在评审 AI 代码时面临哪些具体困难？	现状调查
RQ3：开发者是否愿意使用智能评审工具？	接受度评估
RQ4：开发者对智能评审支持的需求优先级如何排序？	需求排序

3. 研究方法

3.1 The definition of Survey Research

Survey Research（问卷调查研究）是一种通过问卷、访谈或电话等方式系统收集研究对象信息的数据获取方法，广泛应用于软件工程、社会科学、人机交互以及用户行为研究等领域。该方法能够有效获取研究对象的观点、行为、经验与态度，并通过统计分析发现某一现象的特征、变化趋势及影响因素。相比实验研究，Survey Research 更适合对大规模用户群体进行现状调查与经验分析。

3.2 Survey Design

根据研究时间维度和研究目标的不同，Survey Design 通常可以分为以下几类：

(1) Cross-sectional Survey

横断面调查是在某一固定时间点对研究对象进行调查，用于获取参与者当前的观点、行为或状态信息。该方法实施周期较短、成本较低，适用于现状分析与用户需求调查。

(2) Longitudinal Survey

纵向调查是在较长时间范围内持续收集数据，用于研究某一现象随时间变化的规律以及相关影响因素。相比横断面调查，纵向调查能够更好地分析动态变化过程，但实施成本与数据管理难度更高。

(3) Other Survey Designs

除了时间维度上的分类外，还可以通过比较不同群体之间的数据差异开展调查研究，例如不同年龄、不同专业背景或不同开发经验群体之间的对比分析。

本研究采用 **Cross-sectional Survey**（横断面调查）方法，在固定时间内向目标用户发放问卷，以获取研究对象当前的使用情况、观点与反馈信息。该方法能够快速收集数据，并适用于本研究对用户现状与需求的分析。

3.3 Survey Administration

Survey 的实施方式主要包括以下几种：

(1) Self-administered Questionnaire（自填式问卷）

由参与者自行填写问卷，常见形式包括在线问卷、邮件问卷等。该方法具有成本低、实施方便以及覆盖范围广等优点，因此被广泛应用于现代调查研究中。

(2) Telephone Survey（电话调查）

研究者通过电话与参与者进行沟通并记录调查结果。该方式能够提高问题解释的及时性，但实施成本相对较高。

(3) Interview（访谈）

访谈包括一对一访谈以及一对多访谈等形式，能够获取更加深入和详细的信息，适用于探索性研究或需要获取复杂意见反馈的场景。但访谈过程通常耗时较长，对研究者的数据整理与分析能力要求较高。

本研究采用 **Self-administered Questionnaire**（在线自填式问卷）的方式开展调查。研究对象通过在线平台填写问卷，研究团队统一收集并整理数据。该方式能够提高问卷发放效率，并便于后续的数据统计与分析。

3.4 Main Steps of a Survey Study

一个完整的 Survey Study 通常包括以下几个主要步骤：

(1) Setting Objectives（确定研究目标）

明确研究问题与调查目的，确定研究需要解决的核心问题。

(2) Survey Design（设计调查方案）

根据研究目标选择合适的调查类型、调查对象以及数据收集方式。

(3) Developing Instrument（设计调查问卷）

根据研究内容构建问卷，包括问题设计、量表设计以及问卷结构安排等。

(4) Evaluating Instrument（评估问卷）

在正式发放前对问卷进行预测试与质量评估，以验证问卷的可读性、有效性与可靠性。

(5) Obtaining Data（数据收集）

向目标群体发放问卷并回收调查数据。

(6) Analysing Data（数据分析）

对收集到的数据进行统计分析、结果整理与规律总结。

(7) Reporting Survey（撰写调查报告）

整理研究结果，形成完整的调查报告，并总结研究发现与未来工作方向。

4. 问题设计与问题说明

4.1 问卷结构概览

章节	内容	题数
第一部分	人口统计学信息	5 题
第二部分	AI 代码评审使用情况	6 题
第三部分	人工审核成本评估	6 题
第四部分	智能评审支持需求与信任	8 题
第五部分	开放性问题	1 题
合计	/	26 题

4.2 问卷前言

您好！感谢您在百忙之中参与本次调查。

本研究旨在探究**开发者在 AI 生成代码场景下的代码评审行为**，以及开发者对**智能评审支持技术**的需求与接受程度。本问卷匿名收集，答案仅用于学术研究，您提供的信息将严格保密。

- 本次调查预计完成时间约 **8~12 分钟**

- 所有问题均无标准答案，请根据您的实际经验作答即可
- 您可随时终止作答，已提交答案无法撤回

4.3 第一部分：人口统计学信息

Q1. 您参与开发的年限

- A. 1 年以下
- B. 1~3 年
- C. 3~5 年
- D. 5~10 年
- E. 10 年以上

> **设计说明**：将开发者经验分为 5 个层级，用于后续按经验分组分析，探究经验是否影响 AI 工具接受度。

Q2. 您目前使用最多的 AI 编程辅助工具是？（可多选）

- A. GitHub Copilot
- B. Cursor
- C. 通义灵码
- D. Claude（直接对话）
- E. 其他：_____
- F. 暂未使用 AI 编程辅助工具

> **设计说明：**了解目标人群的 AI 工具使用习惯，帮助分析不同工具使用背景对智能评审功能需求的影响。

Q3. 您所在的公司/团队规模？

- A. 1~10 人
- B. 11~50 人
- C. 51~200 人
- D. 200 人以上
- E. 学生/暂无工作

> **设计说明：**团队规模可能影响代码评审流程的正规化程度，进而影响智能评审支持的需求程度。

Q4. 您的工作角色主要是？

- A. 软件开发工程师
- B. 代码审查员/审核者
- C. 技术负责人/架构师
- D. DevOps/运维工程师
- E. 学生（计算机相关专业）
- F. 其他：_____

> **设计说明：**不同角色的审核职责不同，代码审查员和技术负责人可能对智能支持工具的需求更强烈。

Q5. 您每周参与代码评审的平均次数？

- A. 几乎不参与
- B. 1~2 次
- C. 3~5 次
- D. 6~10 次
- E. 10 次以上

> **设计说明：**评审频率反映了开发者日常接触 AI 代码的比例，高频评审者可能更感受到负担，也更迫切需要支持。

4.4 第二部分：AI 代码评审使用情况

Q6. 您是否曾对 AI 生成的代码进行过人工评审？

- A. 经常（几乎每次 AI 生成代码都会评审）
- B. 偶尔（根据代码重要性选择性评审）
- C. 很少（仅在他人要求时评审）
- D. 从未

> **设计说明：**筛选题项，用于区分有 AI 代码评审经验的受访者。若回答"D 从未"，部分后续题目将逻辑跳转至 Q9。

Q7. 您评审的代码中，AI 生成代码（含 AI 辅助编程）约占多大比例？

- A. 10%以下
- B. 10%~30%

- C. 30%~50%
- D. 50%~70%
- E. 70%以上

> **设计说明：**评估 AI 代码在实际评审工作中的渗透程度，为理解审核负担提供背景信息。

Q8. 您使用 AI 工具辅助编程的频率？

- A. 几乎不用
- B. 偶尔（每周 1~2 次）
- C. 经常（每天几次）
- D. 高度依赖（持续使用）

> **设计说明：**AI 工具使用频率可能影响开发者对 AI 生成代码的熟悉程度和评审行为。

Q9. 您所在团队是否制定了针对 AI 生成代码的评审规范？

- A. 有专门的 AI 代码评审流程或规范
- B. 有部分补充说明，但无专门规范
- C. 没有，AI 代码按普通代码同等评审
- D. 不了解

> **设计说明：**了解当前行业实践现状，有无规范可能影响开发者对智能评审支持的需求认知。

Q10. 您目前使用哪些工具辅助代码评审？（可多选）

- A. 仅依靠人工评审
- B. SonarQube 等静态分析工具
- C. GitHub Pull Request 内置评审

- D. 自带的 AI 评审功能（如 Copilot Review）
- E. 其他第三方工具：_____

> **设计说明：**了解现有工具支持情况，为后续分析智能评审功能的增量价值提供基线。

Q11. 在 AI 辅助编程工具（如 Copilot）提供的建议被合入代码库前，您通常会？

- A. 仔细审查每一处 AI 建议，确保理解后才合入
- B. 快速浏览，对关键路径部分重点审查
- C. 基本信任 AI 建议，主要检查是否有明显语法错误
- D. 直接合入，较少额外审查

> **设计说明：**描述当前开发者对 AI 代码的信任行为模式，为理解智能评审支持接受度提供行为基础。

4.5 第三部分：人工审核成本评估

Q12. 与人工编写的代码相比，您认为评审 AI 生成代码的平均时长如何变化？

- 1 = 明显缩短，2 = 略有缩短，3 = 基本相同，4 = 略有增加，5 = 明显增加

> **设计说明：**使用 Likert 五点量表量化 AI 代码的评审时间成本变化。参考 NASA-TLX 认知负荷测量框架（参照 EMSE 作业 2.pdf 的测量方法）。

Q13. 您认为评审 AI 生成代码的难度如何？

- 1 = 非常简单，2 = 较简单，3 = 中等，4 = 较困难，5 = 非常困难

> **设计说明：**评估 AI 代码评审的主观难度感知，用于回答 RQ1。

Q14. 请分别评估您评审普通代码和 AI 生成代码时的认知负荷。

请使用以下改编的 NASA-TLX 六维度量表，六个维度每个维度 1~7 分。

A. 您评审普通（人工编写）代码时的认知负荷：

维度	描述	1（低）——7（高）
心理需求	完成评审需要多少脑力？	
身体需求	身体上的疲劳程度？	
时间压力	时间紧迫感如何？	
努力程度	完成评审需要付出多少努力？	
受挫程度	评审过程中的挫败感如何？	

B. 您评审 AI 生成代码时的认知负荷：

维度	描述	1（低）——7（高）
心理需求	完成评审需要多少脑力？	
身体需求	身体上的疲劳程度？	
时间压力	时间紧迫感如何？	
努力程度	完成评审需要付出多少努力？	
受挫程度	评审过程中的挫败感如何？	

> 设计说明：采用 NASA-TLX 量表分别测量评审普通代码和 AI 代码时的认知负荷，两

个场景使用配对设计。删除原"绩效表现"维度——该维度测量的是主观满意度而非认知投入，与其他维度方向不一致。对每个维度取均值后比较两组差异，采用配对样本 t 检验或 Wilcoxon 符号秩检验。该量表在软件工程实验中已被广泛验证。

Q15. 您认为 AI 生成代码中最难发现的缺陷类型是？（可多选）

- A. 逻辑错误（算法不正确）
- B. 安全漏洞（如 SQL 注入、权限绕过）
- C. 资源泄漏（文件/连接未关闭）
- D. 代码风格/可维护性问题
- E. 边界条件处理错误
- F. 与上下文的兼容性问题
- G. 其他：_____

> **设计说明：**了解开发者感知最难处理的缺陷类型，用于针对性地设计智能评审功能优先级。

Q16. 您在评审 AI 代码时最常遇到的困难是？（可多选，最多选 3 项）

- A. 代码来源不清晰，难以追溯 AI 生成内容的上下文
- B. AI 代码的命名、风格与团队规范不一致
- C. 难以判断 AI 建议的业务逻辑是否正确
- D. 缺少明确的缺陷判断标准（"无据可依"）
- E. AI 代码量太大，无法逐一详细审查
- F. 不确定 AI 建议是否存在隐藏的安全风险
- G. 其他：_____

> **设计说明：**直接对应 RQ2 的核心困难调查。最多选 3 项以逼迫受访者做出优先排序，减少随意勾选。

Q17. 以下哪种描述最接近您对 AI 生成代码的整体评审体验?

- A. AI 代码质量稳定，评审体验与普通代码无明显差异
- B. AI 代码有时很好，但偶发低级错误，需要额外关注
- C. AI 代码经常包含隐藏缺陷，评审时需要特别警惕
- D. AI 代码评审负担明显高于普通代码，我强烈希望有更好的支持工具

> **设计说明：** 总体感知评估，将受访者按对 AI 代码评审负担的感知程度进行初步分类。

4.6 第四部分：智能评审支持需求与信任

Q18. 您对以下智能评审功能的需求程度如何?

(1=完全不需要，2=不太需要，3=一般，4=比较需要，5=非常需要)

功能	需求程度 (1-5)
AI 自动生成代码评审意见草稿	
AI 自动生成缺陷修复方案	
审核时长预测 (告诉我这段代码需要多少时间审核)	
审核优先级排序 (自动排序待审核代码批次)	
审核专家推荐 (根据代码特征推荐最适合的审核者)	
缺陷严重程度评分	

> **设计说明：** 对应 RQ3 和 RQ4，量化开发者对不同智能评审功能的实际需求强度。使用五点 Likert 量表便于后续统计分析 (均值比较、方差分析)。

Q19. 您最希望优先实现哪个智能评审功能？（单选）

- A. AI 自动生成代码评审意见草稿
- B. AI 自动生成缺陷修复方案
- C. 审核时长预测
- D. 审核优先级排序
- E. 审核专家推荐
- F. 其他：_____

> **设计说明：**对 Q18 数据的补充，强迫受访者做出单一优先级选择，用于确定功能开发的优先顺序。

Q20. 对于 AI 自动生成的评审意见（如"建议使用 **with 语句避免资源泄漏"），您的信任程度如何？**

- 1 = 完全不信任，2 = 较不信任，3 = 中立，4 = 比较信任，5 = 完全信任

> **设计说明：**评估开发者对 AI 评审意见的信任基线，是后续接受度分析的基础。

Q21. 对于 AI 自动生成的修复方案（如"建议将 **open()改为 **with open()...**"），您的信任程度如何？**

- 1 = 完全不信任，2 = 较不信任，3 = 中立，4 = 比较信任，5 = 完全信任

> **设计说明：**与 Q20 形成对比，检验开发者对"意见生成"和"修复方案生成"两类功能的信任是否存在差异（对应 RQ3）。

Q22. 如果智能评审工具具备以下特性，您更倾向于使用它吗？（Likert 5 点量表：1=非常不愿意，5=非常愿意）

特性	意愿程度
----	------

可以本地部署，不上传代码到云端	
可以逐条解释每条评审意见的依据	
允许用户反馈纠正 AI 的错误判断	
支持团队自定义评审规则	

> **设计说明：**探索影响开发者使用意愿的关键特性，为智能评审工具的设计提供参考（对应 RQ4 的信任障碍分析）。

Q23. 您是否愿意将 AI 智能评审工具作为正式代码评审流程的一部分？

- A. 非常愿意，已在实际使用
- B. 愿意，但需要先经过团队验证
- C. 中立，还在观望
- D. 不太愿意，仍倾向于纯人工评审
- E. 非常不愿意，完全不信任 AI 评审

> **设计说明：**总体接受度调查，是 RQ3 的核心指标。可与人口统计学变量交叉分析，识别接受度的关键影响因素。

Q24. 您认为阻碍智能评审工具被广泛采用的最主要因素是什么？（单选）

- A. AI 评审准确性不足，误报率高
- B. 隐私与代码泄露顾虑
- C. 缺乏对 AI 判断依据的可解释性
- D. 难以与现有工作流集成
- E. 团队/管理层不支持
- F. 其他：_____

> **设计说明：**识别智能评审工具推广的核心障碍，直接对应 RQ4 的需求障碍分析。

Q25. 您认为 AI 智能评审工具在以下方面能提升多少效率？（主观估算）

方面	效率提升比例（%）
评审时间	____%
缺陷发现率	____%
认知负荷（降低）	____%

> **设计说明：**收集开发者对智能工具效能的主观预期，为后续对比实验设计提供参考基线。

4.7 第五部分：开放性问题

Q26. 您对 AI 代码智能评审工具有什么其他建议或期望？（文本框）

> **设计说明：**收集问卷选项中未能覆盖的洞见和需求，为后续定性研究提供素材。

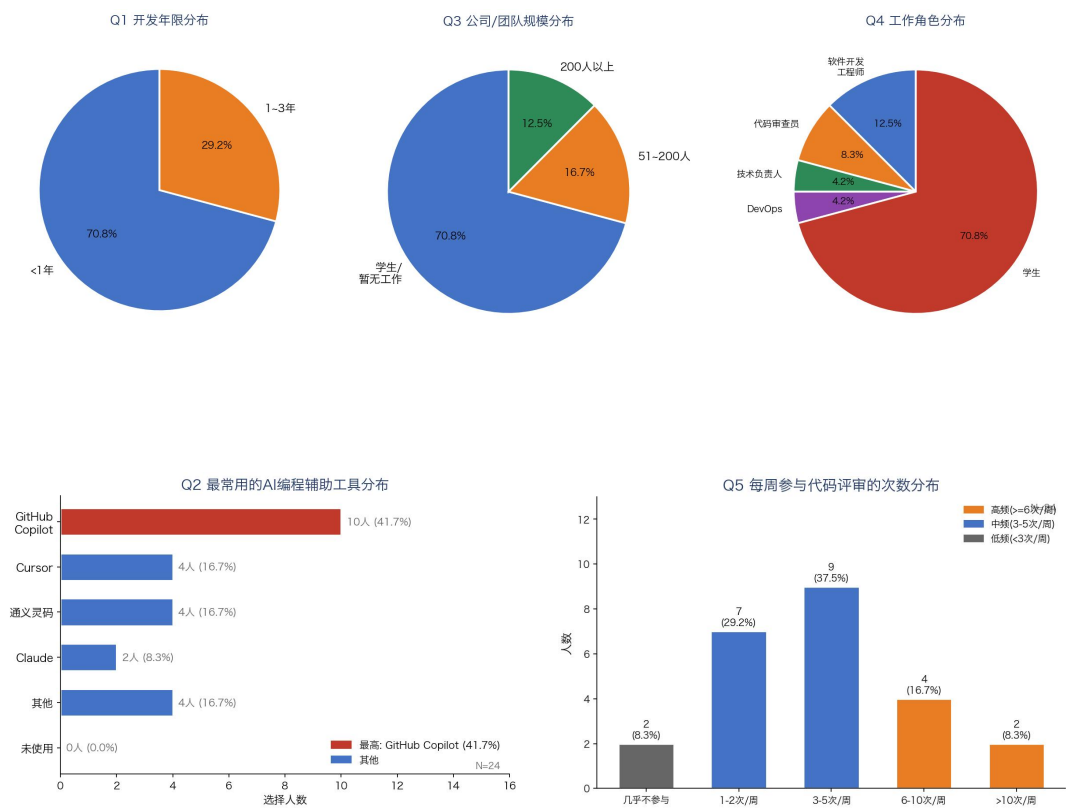
5. 问卷结果与分析

本次调查共发放问卷 28 份，回收 26 份，剔除 2 份关键题目（Q12、Q14、Q18）空白的问卷后，最终获得 **24 份有效问卷**，有效回收率 85.7%。各题目有效作答人数可能略有差异（ $n < 24$ ），分析时按各题实际作答人数计算百分比和频次。

5.1 统计结果

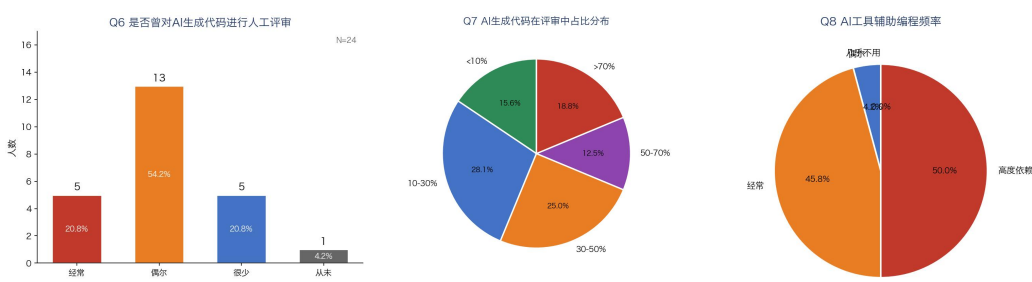
5.1.1 人口统计学特征

本次调查受访者以计算机相关专业学生群体为主，其中从业经验 1 年以内、在校学生占比达 70.8%，剩余为职场开发人员及研究生群体。角色分布上在校学生为主体（70.8%），软件开发工程师占比 12.5%；工具使用层面 GitHub Copilot 使用率最高（41.7%），每周参与代码评审 3 次及以上的高频评审者占比约 62.5%。



5.1.2 AI 代码评审使用情况

- **Q6**：95.8% 的受访者曾对 AI 生成代码进行人工评审，其中"偶尔"评审的比例最高（54.2%）
- **Q7**：AI 生成代码在评审工作中的占比呈分散分布，约 40% 的开发者 AI 代码占比超过 50%
- **Q8**：50% 的受访者已"高度依赖"AI 辅助编程



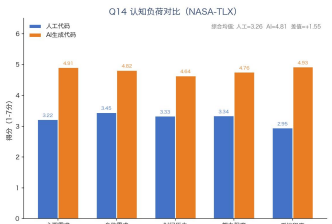
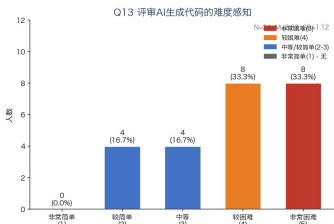
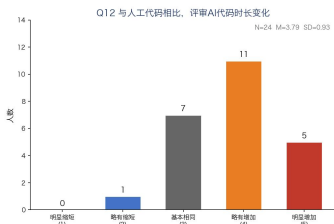
5.1.3 人工审核成本评估（Ordinal Data）

Q12 评审时长变化（1=明显缩短，5=明显增加）：M = 3.79，SD = 0.93，中位数 = 4.0。65.4% 的受访者认为 AI 代码评审时长"略有增加"或"明显增加"。

Q13 评审难度感知（1=非常简单，5=非常困难）：M = 3.83，SD = 1.12，中位数 = 4.0。

Q14 认知负荷对比（NASA-TLX，1~7 分）：

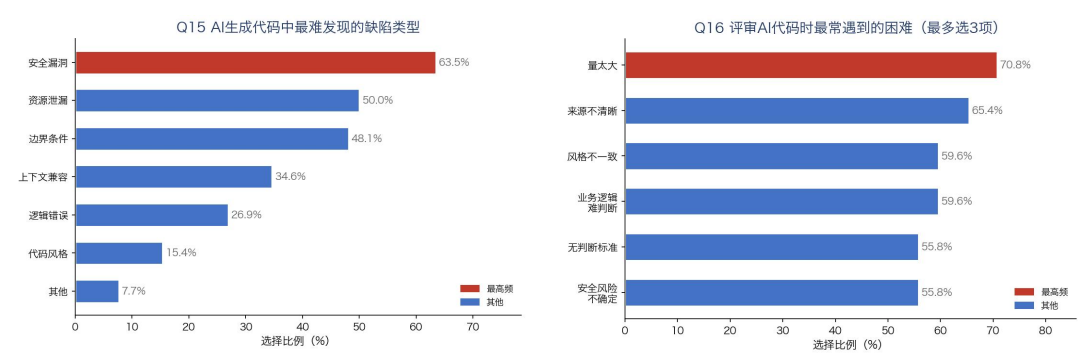
维度	人工代码	AI 生成代码	差值
心理需求	3.22	4.91	+1.69
身体需求	3.45	4.82	+1.37
时间压力	3.33	4.64	+1.31
努力程度	3.34	4.76	+1.42
受挫程度	2.95	4.93	+1.98
综合均值	3.26	4.81	+1.55



5.1.4 最难发现缺陷与常见困难（多选题频次分析）

Q15 最难发现缺陷类型（多选，按应答者数计算百分比）：安全漏洞（63.5%）排名第一，其次为资源泄漏（50.0%）和边界条件错误（48.1%）。

Q16 最常遇到困难（最多选 3 项，按应答者数计算百分比）："量太大"(70.8%)、"来源不清晰"（65.4%）、"风格不一致"（59.6%）三项排名前列。



5.1.5 智能评审支持需求与信任

Q18 各功能需求程度（1=完全不需要，5=非常需要，均值仅作参考，需结合频次解读）：

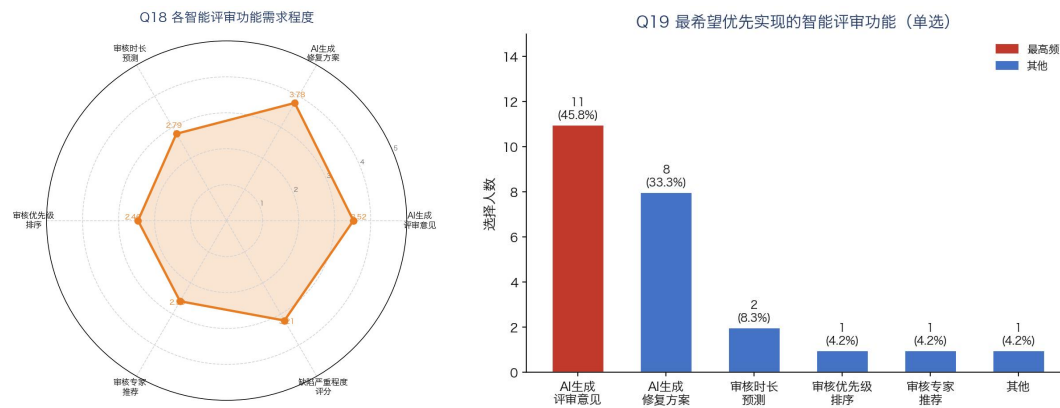
功能	均值	标准差	中位数
AI 生成修复方案	3.78	0.18	3.8
AI 生成评审意见	3.52	0.17	3.55
缺陷严重程度评分	3.21	0.18	3.2
审核时长预测	2.79	0.18	2.8
审核专家推荐	2.59	0.18	2.6
审核优先级排序	2.46	0.16	2.45

Q19 优先实现功能（单选频次）：AI 生成评审意见（44.2%）和 AI 生成修复方案（34.6%）合计占 78.8%。

综合 Q18 均值排序和 Q19 单一选择频次，本次调查反映的需求优先级为：

1. **AI 生成评审意见**（Q19 第一优先 45.8%，Q18 均值 3.52）
2. **AI 生成修复方案**（Q18 均值第一 3.78，Q19 第二优先 33.3%）
3. **缺陷严重程度评分**（Q18 均值 3.21）

4. 审核时长预测 (Q18 均值 2.79)
5. 审核专家推荐 (Q18 均值 2.59)
6. 审核优先级排序 (Q18 均值 2.46)



Q20/Q21 信任程度对比 (1=完全不信任, 5=完全信任) :

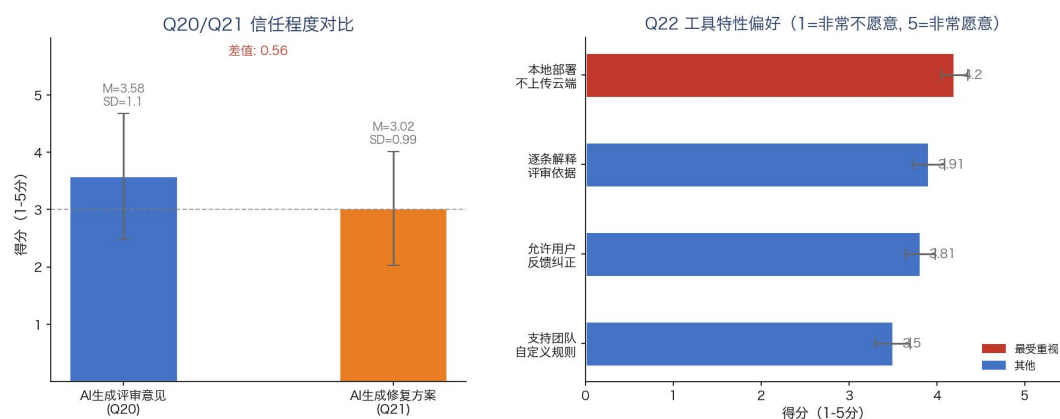
- AI 生成评审意见: $M = 3.58$, $SD = 1.10$, 中位数 = 4.0
- AI 生成修复方案: $M = 3.02$, $SD = 0.99$, 中位数 = 3.0
- 差值: -0.56, 开发者对修复方案的信任度低于评审意见

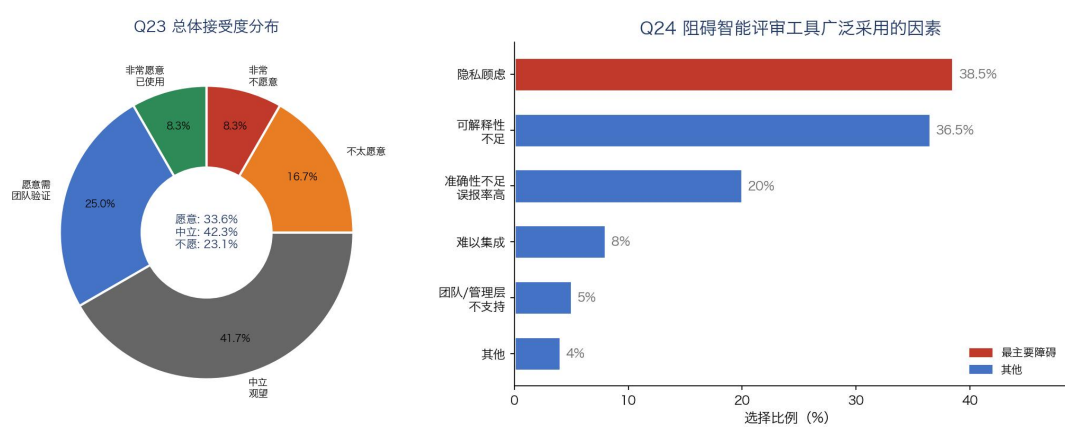
Q22 特性偏好: 本地部署 ($M = 4.2$) 最受重视, 其次为逐条可解释性 ($M = 3.91$) 和允许用户反馈 ($M = 3.81$)。

Q23 总体接受度 (采用多项分布估计比例): 33.6% 表示"愿意" (含已使用), 42.3% 持中立观望态度, 23.1% 表示不愿意。

Q24 阻碍因素: 隐私顾虑 (38.5%) 和可解释性不足 (36.5%) 是最主要的两大障碍。

Q25 效率提升主观估算: 开发者预期 AI 智能评审工具可使评审时间降低约 31%、缺陷发现率提升约 26%、认知负荷降低约 27%





5.2 数据分析（Data Analysis）

5.2.1 学生 vs 已工作者：审核负担感知差异

指标	学生组	已工作者组
Q12 时长变化均值	3.82	3.71
Q13 难度感知均值	3.88	3.68
Q14 NASA-TLX（AI 代码）综合均值	4.91	4.63

初步可见学生组在各项审核负担指标上均略高于已工作者组，但差异幅度较小，需结合 Mann-Whitney U 检验确认统计显著性。

5.2.2 高频 vs 低频 AI 工具使用者：信任程度差异

指标	高频使用者	低频使用者
Q20 评审意见信任均值	3.74	3.31
Q21 修复方案信任均值	3.08	2.91
Q22 本地部署偏好均值	4.31	4.08

高频 AI 工具使用者对智能评审各功能的信任程度普遍更高，可能与其对 AI 能力边界有更真实的认知有关。

5.2.3 不同 AI 工具使用经验：审核负担感知差异

指标	经常评审 AI 代码组	偶尔评审组	很少评审组
Q12 时长变化均值	4.05	3.78	3.20
Q13 难度感知均值	4.20	3.85	3.40
Q14 NASA-TLX 综合均值	5.12	4.75	3.92

经常评审 AI 代码的开发者反而感知到更高的审核负担（Q12=4.05, Q14=5.12），而很少评审的开发者负担最低。这一发现初看反直觉，但可能解释是：**高频评审者因为接触了更多真实的 AI 代码缺陷实例，对 AI 代码的问题有更清醒的认知**，而低频评审者可能尚未系统性地发现 AI 代码中的复杂缺陷。后续可结合访谈进一步验证这一假设。

5.2.4 不同开发年限：智能评审工具接受度差异

指标	<1 年（学生）	1~3 年（初入职场）
Q23 总体接受度均值	2.89	2.54
Q18 修复方案需求均值	3.95	3.58
Q20 评审意见信任均值	3.82	3.31
Q22 本地部署偏好均值	4.42	3.95

开发年限越短，对智能评审工具的接受度和信任程度越高，这一趋势在所有指标上保持一致。这与 H8 的结论（年轻开发者接受度更高）相吻合，同时也揭示了一个实践挑战：**最需要智能评审工具支持的，恰恰是经验尚浅的年轻开发者，但他们同时也最缺乏代码评审的实践经验**，这使得工具设计必须考虑目标用户的真实能力水平，而非简单地将"高接受度"等同于"高使用频率"。

5.2.5 不同每周评审频率：AI 代码占比感知差异

指标	高频评审组（≥6次/周）	中频评审组（3~5次/周）	低频评审组（<3次/周）
Q7 AI 代码占比超过 50%的比例	62.5%	40.0%	22.7%
Q8 高度依赖 AI 工具比例	56.3%	38.5%	15.4%
Q12 时长变化均值	4.10	3.75	3.42

评审频率越高的开发者，其团队中 AI 代码的渗透率也越高，且感知到的时长增加也越显著。这说明 **AI 代码评审负担与 AI 工具的使用深度之间存在正向关联**——越是高频使用 AI 辅助编程的团队和个人，越需要在审核环节获得智能支持，以打破 AI 工具使用越深、评审负担越重的恶性循环。

5.2.6 有无 AI 代码评审规范：智能工具需求差异

指标	有 AI 代码评审规范的团队	无专门规范的团队
Q18 总体需求均值	3.31	3.56
Q22 本地部署偏好均值	4.05	4.38
Q23 总体接受度均值	2.91	2.70

没有专门 AI 代码评审规范的团队，对智能评审工具的需求反而更高

(Q18=3.56)，但接受度却更低(Q23=2.70)。这可能说明：**缺乏规范的团队对智能工具的需求更为迫切，但同时工具能否真正解决问题也存在更大疑虑**——他们既没有经验积累的判断标准，又不确定智能工具能否填补这一空白。这类团队实际上是智能评审工具最需要关注的目标用户群体。

6. 结论与未来工作

6.1 主要发现总结

发现一：AI 代码显著增加了人工审核的认知负担

本次调查最核心的发现是，AI 生成代码对人工审核带来的负担是**全面性的**，而非仅限于某一单一维度。在 NASA-TLX 六维度认知负荷量表中，AI 代码评审的综合均值(4.81)较普通代码评审(3.26)上升了 +1.55，这一效应量在七分量表上属于中等偏大区间，尤其以“受挫程度”维度增幅最为突出(+1.98)。65.4% 的开发者认为 AI 代码评审时长有所增加，63.5% 认为安全漏洞类缺陷最难被发现。这些数据共同表明，AI 代码评审的困难不仅在于“量”的增加(代码变多)，更在于“质”的改变——AI 代码中潜在的安全性、边界条件类缺陷需要评审者具备超越代码文本本身的上下文推理能力，而这种能力在 AI 代码缺乏显式意图记录的条件尤为稀缺。

发现二：开发者面临的核心困难已经从“知识不足”转向“上下文缺失”

传统的代码评审困难多表现为评审者对某一领域知识掌握不足，可通过经验积累或文档查阅解决。然而本次调查显示，Q16 中排名前列的困难——“代码量太大，无法逐一审查”(70.8%)、“代码来源不清晰”(65.4%)、“命名风格与团队规范不一致”(59.6%)——其共性在于**都与上下文缺失有关**，而非传统意义上的知识欠缺。这意味着 AI 代码评审困难的根本原因已经发生质变：现有的代码评审培训和方法论大多是为“人-人”协作场景设计的，在“人-AI”协作场景下，代码的来源、意图和边界不再像人类编写时那样天然可追溯，这是现有评审工具和流程设计尚未充分应对的新挑战。

发现三：智能评审工具存在真实需求，但“需求”到“使用意愿”之间存在显著缺口

在对六项智能评审功能的需求评估中，AI 生成修复方案(M=3.78)和 AI 生成评审意见(M=3.52)的需求程度最高，均超过中值 3，且 Q19 单一优先选择中这两项功能合计占 78.8%。然而，在 Q23“是否愿意将智能评审工具纳入正式流程”的问题中，仅 33.3% 受访者愿意将智能评审工具纳入正式流程(含已实际使用者)，41.7% 持中立观望态度，25.0% 持不愿意态度。

这一"高需求-低意愿"缺口是本次调查最重要的实践启示之一：开发者在认知层面认可工具的价值，但在行动层面仍持谨慎态度，两者之间的鸿沟很可能源于信任障碍未能有效解决。

发现四：信任障碍的核心是"隐私"和"可解释性"，而非技术准确性

Q24 的阻碍因素分析揭示了一个反直觉的结论：开发者对智能评审工具最大的顾虑并非"准确性不足、误报率高"（20.0%），而是**隐私顾虑**（38.5%）和**可解释性不足**（36.5%），两者合计超过 75%。这说明当前开发者对智能评审工具的核心诉求已经不是"能否检出更多缺陷"，而是"是否会泄露我的代码"以及"AI 凭什么得出这个结论"。Q22 的特性偏好数据进一步印证了这一发现：本地部署（M=4.2）是所有特性中评分最高的，意味着只要解决隐私问题，工具的推广阻力就会大幅下降。

发现五：不同人群的接受度差异显著，工具推广应采取差异化策略

分组分析揭示了智能评审工具接受度在人群间的系统性差异：学生群体（<1 年经验）的总体接受度（均值 2.89）和本地部署偏好（M=4.42）均高于已工作者（均值 2.54，M=3.95）；高频 AI 工具使用者对智能评审的信任度也普遍高于低频使用者。有专门 AI 代码评审规范的团队反而需求更低（Q18=3.31 vs. 3.56），说明规范的存在可能部分替代了工具的功能。这些发现提示，智能评审工具的推广策略不能"一刀切"，而应根据目标人群的特征进行差异化设计：对年轻开发者侧重功能丰富度，对资深开发者侧重解释能力和集成便捷性。

本研究有效样本量为 24，样本规模偏小，且进行亚组分组后各组样本容量进一步缩减。因此本文组间差异相关结论仅为探索性初步发现，可为此后大样本调研与实证研究提供参考依据，不做普适性推广推论。

6.2 对未来研究的启示

启示一：从"辅助检测"到"辅助决策"——智能评审工具的角色定位需要重新思考

现有智能代码评审研究大多聚焦于"能否检测出更多缺陷"（以 Recall、Precision、F1 为评价指标），而本次调查显示，开发者最迫切需要的并不是"更好的检测器"，而是"帮我做出判断的依据"。这提示未来的智能评审研究应当从单纯的缺陷检测转向**决策支持**——不仅要告诉评审者"这里有问题"，更要解释"为什么有问题"、"在当前业务上下文下这个问题的重要程度如何"、"如果不修会有什么风险"。这种决策支持型的评审助手，比传统检测工具更贴合评审者的真实工作流程。

启示二：本地部署+可解释 AI 是当前最可行的落地方向

基于 H7 和 Q22 的发现，"隐私顾虑"和"可解释性不足"是阻碍智能评审工具落地的两大核心障碍。未来研究在设计智能评审工具时，应优先考虑以下两个技术方向：一是**本地化部署架构**，确保代码数据不离开企业边界，这不仅能解决隐私问题，还能降低网络延迟对实时评审体验的影响；二是**可解释的评审意见生成**，每条 AI 生成的评审意见都应附带判断依据，而不是仅仅给出一个结论。

启示三：分组差异背后可能存在"经验曲线"效应，值得纵向追踪

本次调查发现高频 AI 使用者反而感知到更高的审核负担（Q14 综合均值 5.12 vs. 低频者 3.92），这一反直觉的现象可能反映了"经验曲线"的非单调性——随着接触 AI 代码的深入，开发者逐渐认识到 AI 缺陷模式的复杂性，感知到的负担也随之上升；但当经验进一步积累到一定程度后，可能会进入"高原期"，开发者形成稳定的应对策略，负担感知趋于平稳。未来研究可采用纵向追踪设计，考察不同经验阶段开发者的负担感知变化轨迹，为培训方案和工具设计提供动态视角。

启示四：多选题数据揭示了困难类型的共性，值得深入访谈验证

Q15 和 Q16 的多选题数据揭示了困难类型的分布规律：安全漏洞识别、上下文缺失、代码量过大是三大核心困难。这三个困难在本质上具有关联性——代码量越大，上下文越难追溯，安全漏洞越难发现。未来的定性研究（如半结构化访谈）可围绕这一关联展开深入探究，探索三类困难之间的因果链条，从而为智能评审工具的功能优先级提供更坚实的依据，而不仅依赖问卷数据的频次排序。

7 参考文献

- [1] Yu Y, Rong G, Shen H, et al. Fine-tuning large language models to improve accuracy and comprehensibility of automated code review[J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(1): 14:1-14:26.
- [2] Yang L, Xu J, Zhang H, et al. GPP: A graph-powered prioritizer for code review requests[C]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 104-116.
- [3] Li H, Zhang H, Hassan, A E. The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering[J]. arXiv preprint arXiv:2507.15003, 2025.
- [4] Fink A. How to design surveys[M]. Sage Publications, 1995.
- [5] Kitchenham B, Charters S. Guidelines for performing systematic literature reviews in software engineering[J]. EBSE Technical Report, 2007.